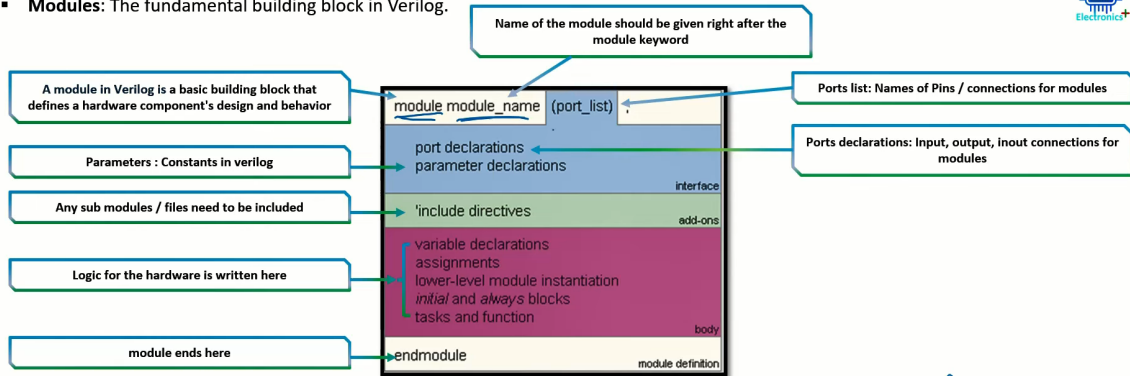


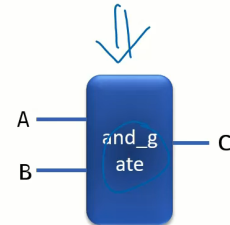
Basic Syntax

- **Modules:** The fundamental building block in Verilog.



// This module takes two inputs and generates one output.

```
module and_gate(input A, input B, output C);
    assign C = A & B;
endmodule
```



Data Types

- ➔ ▪ **Net: Wire**: Represents a continuous value. The default value of the net is Z.
- ➔ ▪ **Reg**: Represents a value stored over time (used in procedural blocks). The default value of the reg is X.
- **Integer**: Stores 32-bit signed integers.
- **Real**: Stores floating-point numbers.

```
module example(input clk, input A, input B, output C);
    wire w;      // wire declaration
    reg r;       // reg declaration

    assign w = A & B; // wire continuous assignment
    always @ (posedge clk) begin
        r = A;      // reg procedural assignment
    end
endmodule
```

can be anything on the right

(not for registers) registers can be there @ right side
 assigning registers assigned when (--) → occurs = 7*
 ex: here every time the transition happens, we assign at the clock edge

→ i.e whenever there is a change in one of the values (reg or wire), trigger the block.

Operators in Verilog

- **Logical Operators:** &, |, ^, ~
- **Relational Operators:** ==, !=, >, <
- **Arithmetic Operators:** +, -, *, /
- **Shift Operators:** <<, >>

```
module adder(input [3:0] A, input [3:0] B,
output [3:0] Sum, output Cout);

assign {Cout, Sum} = A + B;

endmodule
```

Logical Operators	Reduction Operators	Relational Operators
& bitwise AND	& reduce via AND	== equal
bitwise OR	~& reduce via NAND	!= not equal
^ bitwise XOR	reduce via OR	> greater than
^^ bitwise XNOR	~ reduce via NOR	>= greater than or equals
~ bitwise NOT	~ reduce via XOR	< less than
&& boolean AND	^^ reduce via XNOR	<= less than or equals
boolean OR	Shift Operators	Other Operators
! boolean NOT	>> shift right	{ } concatenate
	<< shift left	{N{ } replicate N times
Arithmetic Operators	>>> arithmetic shift right	
+ addition		
- subtraction		

Assignment types in verilog

Types	Continuous assignment	Procedural assignment
Where it's used	Outside of procedural blocks	Inside of procedural blocks
What it does	Drives values onto nets	Updates register and memory data types
When it happens	Automatically active at time zero	Evaluated when the statement is encountered
How it works	Continuous assignments automatically update the variable whenever the right-hand side operand changes. example, assign a = b & c	assignments made within procedural blocks like "always" or "initial" "blocking" or "non-blocking" depending on their execution behavior within a block of code

assign always a wire

always a register

Descriptions in verilog

Different levels of abstraction and methodologies / styles used to describe hardware

Gate-Level Description :

- model the design using logical gates (like AND, OR, NOT).
- Verilog supports built-in primitive gates modeling.
- This is a more detailed and explicit description, closely mimicking hardware components.

```
module MUX2to1 (
input a, // First input
input b, // Second input
input sel, // Selector input
output y // Output
);
wire not_sel, and1, and2;

// NOT gate
not (not_sel, sel);

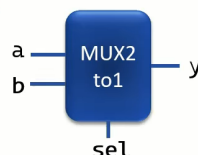
// AND gates
and (and1, a, not_sel);
and (and2, b, sel);

// OR gate to combine the results
or (y, and1, and2);
endmodule
```

Dataflow Description :

- uses assign statements to model the flow of data through the circuit.
- Mainly used to describe combinational circuits
- It's an intermediate level of abstraction between behavioral and gate-level designs.

```
module MUX2to1 (
input a, // First input
input b, // Second input
input sel, // Selector input
output y // Output
);
// Dataflow description using a
conditional expression
assign y = (sel == 1'b1) ? b : a;
endmodule
```



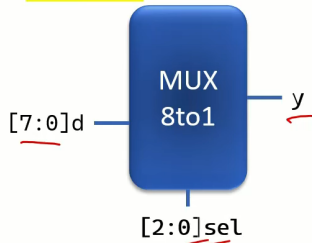
sel	y
0	a
1	b

Behavioral Description:

- Describes the functionality of the hardware using a high-level approach with always blocks and if-else, case and other statements.
- Used to describe complex circuits (primarily sequential)

```
module MUX2to1 (
input a, // First input
input b, // Second input
input sel, // Selector input
output reg y // Output (declared
as reg because it will be assigned
in always block)
);
always @ (sel or a or b) begin
if (sel)
y = b; // If sel is 1,
output b
else
y = a; // If sel is 0,
output a
end
endmodule
```

8:1 Mux



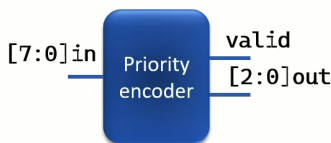
sel	y
000	d[0]
001	d[1]
010	d[2]
011	d[3]
100	d[4]
101	d[5]
110	d[6]
111	d[7]

```
module mux_8to1_behavioral (
    input wire [7:0] d, // 8 data inputs
    input wire [2:0] sel, // 3-bit select input
    output reg y // Output
);
```

```
    always @(*) begin
        case (sel)
            3'b000: y = d[0];
            3'b001: y = d[1];
            3'b010: y = d[2];
            3'b011: y = d[3];
            3'b100: y = d[4];
            3'b101: y = d[5];
            3'b110: y = d[6];
            3'b111: y = d[7];
            default: y = 1'b0;
        endcase
    end
endmodule
```

The "wire" can be neglected. They are wires by default. * => all input variable are considered; trigger when anyone changes

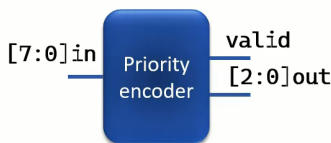
8to3 Priority Encoder : using if else



[7:0]In	out	valid
10000000	111	1
01000000	110	1
00100000	101	1
00010000	100	1
00001000	011	1
00000100	010	1
00000010	001	1
00000001	000	1
00000000	XXX	0

```
module Priority_Encoder_8to3 (
    input [7:0] in, // 8-bit input
    output reg [2:0] out, // 3-bit output (encoded value)
    output reg valid // Valid signal: indicates if at least one input is active
);
    always @(*) begin
        // Check the highest priority input first
        if (in[7]) begin
            out = 3'b111; valid = 1; // Input 7 is active
        end
        else if (in[6]) begin
            out = 3'b110; valid = 1; // Input 6 is active
        end
        else if (in[5]) begin
            out = 3'b101; valid = 1; // Input 5 is active
        end
        else if (in[4]) begin
            out = 3'b100; valid = 1; // Input 4 is active
        end
        else if (in[3]) begin
            out = 3'b011; valid = 1; // Input 3 is active
        end
        else if (in[2]) begin
            out = 3'b010; valid = 1; // Input 2 is active
        end
        else if (in[1]) begin
            out = 3'b001; valid = 1; // Input 1 is active
        end
        else if (in[0]) begin
            out = 3'b000; valid = 1; // Input 0 is active
        end
        else begin
            out = 3'bXXX; // Default output value if no input is active
            valid = 0; // Default valid signal as 0
        end
    end
endmodule
```

8to3 Priority Encoder : using casex



In [7:0]	out	valid
10000000	111	1
01000000	110	1
00100000	101	1
00010000	100	1
00001000	011	1
00000100	010	1
00000010	001	1
00000001	000	1
00000000	XXX	0

- Casex statement treats x and z as dont cares.
- x will be matched to 0 or 1 or z or x and z will be matched to 0 or 1 or x or z.

```
module Priority_Encoder_8to3 (
    input [7:0] in, // 8-bit input
    output reg [2:0] out, // 3-bit output (encoded value)
    output reg valid // Valid signal: indicates if at least one input is active
);
    always @(*) begin
        casex (in) // 'x' allows wildcards, any undefined ('x') or high-impedance ('z') values are ignored
            8'b1xxxxxx: out = 3'b111; // Input 7 is active
            8'b01xxxxxx: out = 3'b110; // Input 6 is active
            8'b001xxxxx: out = 3'b101; // Input 5 is active
            8'b0001xxxx: out = 3'b100; // Input 4 is active
            8'b00001xxx: out = 3'b011; // Input 3 is active
            8'b000001xx: out = 3'b010; // Input 2 is active
            8'b0000001x: out = 3'b001; // Input 1 is active
            8'b00000001: out = 3'b000; // Input 0 is active
            default: out = 3'b000; // No input is active
        endcase
        valid = (in != 8'b00000000); // Set valid to 1 if any input is active
    end
endmodule
```



Synchronous reset and Asynchronous reset

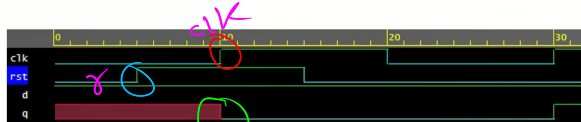
D flipflop using Synchronous reset

```
module d_ff (
    input clk, rst,
    input d,
    output reg q
);

always@(posedge clk)
    //Synchronous reset
    begin
        if(rst) q ≤ 0; //active high reset
        else    q ≤ d;

    end

endmodule
```



Depends/waits till
clk edge

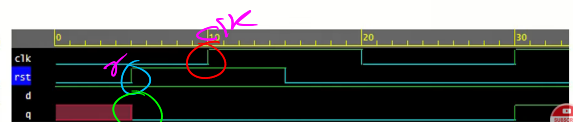
D flipflop using Asynchronous reset

```
module d_ff (
    input clk, rst,
    input d,
    output reg q
);

always@(posedge clk or posedge rst)
    //asynchronous reset
    begin
        if(rst) q ≤ 0; //active high reset
        else    q ≤ d;

    end

endmodule
```



Circuit depends on clk
edge or reset edge.

Blocking Assignment & Non Blocking Assignment



Blocking Assignment

Blocking assignments are represented by the equal sign (=)

Blocking assignments execute in Series

Blocking assignments are used in combinational circuit design

Non Blocking Assignment

Non Blocking assignments are represented by the less than equal sign (<=)

Non-blocking assignments execute in parallel

Non Blocking assignments are used in sequential circuit design

Timing and Delays :



Delay: This is a simple delay that specifies a time delay after an event. It's the most common way to introduce delays in Verilog.

``timescale 1ns / 1ps` // Specifies that the time unit is 1ns, with a precision of 1ps

```
a = b; // Assign 'b' to 'a'
#5;    // Wait for 5 time units
c = a; // Assign 'a' to 'c' after 5 time units
```

`assign #10 y = a & b;` // 'y' is assigned the value of 'a & b' after 10 time units

`and #(10) (out, in1, in2);` // AND gate with 10-time unit delay

```
always @(posedge clk) begin // Trigger on rising edge of 'clk'
    // Action
end
```

```
always @ (posedge clk) begin
    #5 x = y; // x is assigned the value of y with a 5-time unit delay
end
```

Testbench in Verilog

Half Adder

// Design code in dataflow modeling

```
module half_adder(a, b, sum, carry);
```

```
input a, b;
```

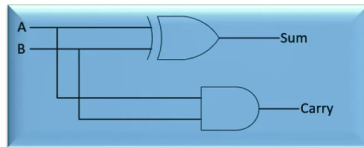
```
output sum, carry;
```

// assign used in dataflow modelling

```
assign carry = a&b;
```

```
assign sum = a^b;
```

```
endmodule
```



// testbench

```
module half_adder_tb;
```

```
reg x,y;
```

```
wire s, c;
```

```
//half_adder dut (x, y, s, c);
```

```
half_adder dut
```

```
(.a(x), .b(y), .sum(s), .carry(c)); //design instantiation
```

```
//stimulus
```

```
initial begin
```

```
    x=0; y=0;
```

```
#10 x=0; y=1;
```

```
#10 x=1; y=0;
```

```
#10 x=1; y=1;
```

```
#20 $finish;
```

```
end
```

```
initial begin
```

```
$monitor( $time, "x=%b y=%b sum=%b  
carry=%b", x,y,s,c);
```

```
end
```

```
endmodule
```



Testbench in Verilog

clk	rst	J	K	Q
1	1	x	x	0
1	0	0	0	Q (hold)
1	0	0	1	0
1	0	1	0	1
1	0	1	1	Q' (toggle)

//Design of JK Flipflop

```
module jk_flipflop
```

```
( input J,
```

```
input K,
```

```
input Clk,
```

```
output reg Q);
```

```
always @(posedge Clk)
```

```
begin
```

```
if (rst) // If reset is high, output Q is reset to
```

```
0
```

```
    Q <= 0;
```

```
else
```

```
case ({J, K})
```

```
    2'b00: Q <= Q;
```

```
    2'b01: Q <= 1'b0;
```

```
    2'b10: Q <= 1'b1;
```

```
    2'b11: Q <= ~Q;
```

```
endcase
```

```
end
```

```
endmodule
```

//Testbench

```
module tb_jk_flipflop;
```

```
// Declare variables for the inputs and output of the JK flip-flop
```

```
reg J, K, Clk, rst;
```

```
wire Q;
```

```
// Instantiate the JK flip-flop
```

```
jk_flipflop uut (
```

```
    .J(J), .K(K), .Clk(Clk), .rst(rst), .Q(Q) );
```

```
// Generate clock signal (with period 10 time units)
```

```
always begin
```

```
    #5 Clk = ~Clk; // Toggle the clock every 5 time units (10 time units period)
```

```
end
```

```
// Test stimulus
```

```
initial begin
```

```
    // Initialize the clock and reset
```

```
    Clk = 0;
```

```
    rst = 0; // Initially, reset is deactivated
```

```
$monitor("%0t\t%b\t%b\t%b\t%b\t%b", $time, J, K, rst, Q);
```

```
// Test Case 1: Apply Reset
```

```
rst = 1; // Active high reset
```

```
#10; // Wait for one clock cycle
```

```
rst = 0; // Deactivate reset
```

```
#10;
```

```
// Test Case 2: J=0, K=0 (No Change)
```

```
J = 0; K = 0;
```

```
#10;
```

```
// Test Case 3: J=0, K=1 (Reset, Q = 0)
```

```
J = 0; K = 1;
```

```
#10;
```

```
// Test Case 4: J=1, K=0 (Set, Q = 1)
```

```
J = 1; K = 0;
```

```
#10;
```

```
// Test Case 5: J=1, K=1 (Toggle, Q will flip)
```

```
J = 1; K = 1;
```

```
#20; $finish;
```

```
end
```

```
endmodule
```



→ always run in -definitely

$$f = \frac{1}{T}$$

3. How it's used

You typically assign values to `reg` types inside procedural blocks like this:

verilog

Copy code

```
always @(posedge clk) begin
    data <= input_data; // non-blocking assignment
end
```

Here:

- Because `data` is assigned inside an `always @(posedge clk)` block, it will synthesize to a **flip-flop**.
- It "remembers" its value between clock cycles — behaving like a hardware register.

Another example:

verilog

Copy code

```
always @(*) begin
    result = a & b; // blocking assignment
end
```

Here:

- `result` is still a `reg`, but this describes **combinational logic**, not a flip-flop.

Clock Generation

`timescale 1ns/1ns;

```
module clock_generator;
  reg clk1;
  reg clk2;
```

```
// clock generation using always, 50% duty cycle
initial clk1=0; //0 ns
  always #5 clk1=~clk1; // Period 10ns, Freq 100MHz
```

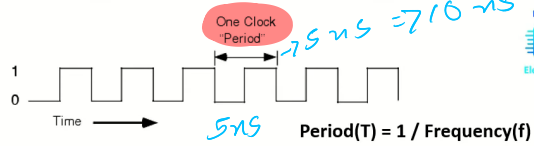
```
// clock generation using forever, 50% duty cycle
```

```
initial begin
  clk1=0; //0 ns
  forever #10 clk1=~clk1; // Period 20ns, Freq 50MHz
end
```

```
// 10% Duty cycle, Period 20ns, Freq 50MHz
always begin
  #9 clk2 = 1 ; //on period
  #1 clk2 = 0 ;// off period
end
```

```
initial
  #100 $finish;
```

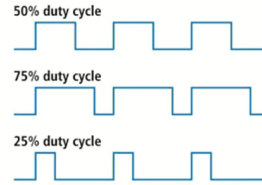
```
endmodule
```



off $\rightarrow$ on

always vs forever

- always is synthesizable
- forever is non synthesizable
- always can be used in module
- forever cannot be used directly, it should be with in procedural block
- always can be used in design & Testbench
- forever used only in testbench



⚡ 4. Key difference from wire

Feature	reg	wire
Can store a value	✓ Yes	✗ No
Assigned in	procedural blocks (always, initial)	continuous assignments (assign)
Represents	variable (flip-flop, latch, or logic node)	physical wire connection
Default value	Unknown (X)	Driven by connected logic

Similar to always
can be used in initial block.